C/C++-Coding Styleguide

-Programmierrichtlinien-

Hochschule Offenburg
Fakultät EMI

Lehrgebiete:

Ingenieur-Informatik (II) / Programmierung 2 (P 2)
Embedded Systems 1+2 (ES 1+2)
Embedded Echtzeitsysteme (EES)
Objektorientierte Softwareentwicklung (OOSWE)

Prof. Dr.-Ing. Daniel Fischer
Dipl.-Ing. (FH) Alexander Badura

Ergeben sich bezüglich den Lehrgebieten unterschiedliche Programmierrichtlinien, werden die beiden Regelvarianten der Richtlinie mit II (Ingenieur-Informatik) und ES (Embedded Systems 1+2, Embedded Echtzeitsysteme) kenntlich gemacht. Programmierrichtlinien, die nur für Embedded Systems 1+2 und Embedded Echtzeitsysteme relevant sind, sind ebenso mit ES gekennzeichnet.
Zur Bestimmung einzelner Metriken kommt das SW-Werkzeug Testwell CMT++ zur Anwendung. Dieses SW-Werkzeug ist in der zu verwendenden VM (BWLehrpool) installiert und dient dazu Metriken, wie z.B. LOCpro (Lines Of Code - program), c% (Kommentardichte) und v(G) (Zyklomatische Komplexität – Cyclomatic Complexity CC - nach McCabe) zu bestimmen. Empfohlene Regeln (recommended) sind in der Nummernspalte grau hinterlegt.

Allgemeines		
A1	Der **Code-Editor** ist so zu konfigurieren, dass ein Tabulator durch **drei** Leerzeichen ersetzt wird. *Damit ist das Erscheinungsbild vom Sourcecode gleich, wenn dieser in verschiedenen Editoren geöffnet wird.*	Innerhalb der IDE können Einstellungen für alle oder für spezifische Sprachen (C/C++) eingegeben werden. **Microsoft Visual Studio:** *Tools → Options → Text-Editor → C/C++ → Tabs → Tab size* und *Indent size auf 3 oder 4, Insert spaces* anwählen. **Keil uVision 5:** *Edit → Configuration → Tab size* auf 3 oder 4 und *Insert spaces for tabs* anwählen. **STM32CubeIDE:** Window -> Prefereneces -> General -> Editors -> Text Editors -S Displayed tab with auf 3 oder 4 stellen sowie Insert spaces for tabs anwählen.

A2	Sourcecode wird in **englischer Sprache** verfasst.
	Das gilt z.B. für Kommentare, Variablen- und Funktionsnamen.
		`// User has to enter the max. value`
		`int iMaxUserValue;`
		`// Check against limits`
		`BOOL IsMaxValue(int iEnteredValue);`

A3	Faustregel: Pro 10 Programmzeilen 1-3 Zeilen Kommentar! (c%: 10-30%)
	Auch leere Zeilen tragen zur Lesbarkeit bei.
	Es sind **keine Trivialitäten** zu dokumentieren. Alles, was sich aus dem Programmcode für einen mittelmäßigen Programmierer sofort ergibt, ist nicht zu dokumentieren!
	In Testwell CMT++ ist die Kommentardichte als c% und die Programmzeilen als LOCpro ausgewiesen. Ist c% zu groß oder zu klein, dann wird ein „-“ angezeigt.
		`// increment i -> trivial, don’t comment this`
		`i++;`
		
		`// Is source of timer interrupt match`
		`// register 2?`
		`if (LPC_TIM0->IR & (1<<2))`

A4	Kommentare beginnen mit einem //. Mehrzeilige Kommentare /* */ sind nicht erlaubt. Ausnahme: doxygen-Kommentare! (siehe Embedded Systems 1)
	Bei projektweiter Suche ist im Suchergebnis sofort ersichtlich, welche Zeile auskommentiert ist und welche nicht.
		`// this is okay`
		
		`/* this is`
		`not`
		`okay */`

A5	Ein C-Projekt ist in viele *.c- und *.h-Dateien aufzuteilen. Dateinamen beginnen mit einem Großbuchstaben.
	Die Wiederverwendbarkeit der Module wird dadurch erhöht (Baukastenprinzip).
		`Nqueensmain.c`
		`Recalgo.c`
		`Recalgo.h`

A6	Eine Funktion darf maximal nur 4-40 Programmzeilen enthalten. *In Testwell CMT++ sind die Programmzeilen als LOCpro ausgewiesen.*	
A7	Eine C-Datei darf maximal nur 4-400 Zeilen enthalten	

A8	Eine C-Datei (Modul) ist wie folgt aufgebaut:
	
	1. Systemheaderdateien (z.B. #include <stdio.h>)
	2. User Headerdateien (z.B. #include "Eightqueen.h")
	
	3. Modulspezifische Datentypen
	Defines, Konstanten, Enumerationen, Structs, Unions, Bitfields, Typedefs, Makros
	
	4. Externe und globale Variablen (Ausnahmefall)
	
	5. Funktionsdeklarationen für alle Funktionen außer main
	
	6. Funktionsimplementierungen
	(Die main-Funktion wird in einem Modul immer als erste Funktion implementiert.)
	Damit ein Modul übersichtlich bleibt, werden teilweise die hier aufgelisteten Elemente in Header-Dateien ausgelagert.
		`#include <stdio.h>`
		`#include <conio2.h>`
		`#include "Eightqueen.h"`
		
		`#define SIZEOFBOARD 8`
		`typedef unsigned short int UINT16_t;`
		`const double cdPI = 3.1415926535;`
		
		`enum Modes`
		`{`
		`  continuous,`
		`  stop`
		`};`
		
		`extern int iSolutionsFound;`
		
		`int SetNextQueen(struct nQueen* psNQueen);`
		`void PrintBoard(struct nqueen* psNQueen);`
		
		`int main(void)`
		`{`
		
		`}`

A9
II
	Keine Verwendung von Variablen mit projekt- oder modulglobaler Gültigkeit. Stattdessen z.B. in main die verwendeten Variablen definieren und diese an andere Funktionen mit „Call by Reference“ übergeben.	`int main(void)`
		`{`
		`  int iValue;`
		`  vfunc1(iValue);  // Call by value`
		`  vfunc2(&iValue),  // Call by reference`
		`}`

A9
ES
	Die Verwendung von Variablen mit projekt- oder modulglobaler Gültigkeit ist zu minimieren. Um ungewünschte Optimierungen zu verhindern ist das Schlüsselwort *volatile* einzusetzen.	`// w: SysTickHandler, main`
		`// r: main`
		`volatile uint32_t ui32Clock1s;`

A10 ES	Projektglobale Variablen müssen in einer c-Datei definiert werden.	`volatile uint32_t ui32Clock1s;`

A11	Projektglobale Variablen können in der c-Datei als modulglobale Variablen gekapselt werden. Der Zugriff erfolgt dann mit Getter- und Setterfunktionen.
	Dadurch können Checkfunktionen (bei den Setter- und ggf. auch bei den Getterfunktionen) realisiert werden.
	Ebenso ergibt sich beim Debuggen ein wesentlicher Vorteil: Setzt man einen Breakpoint in die Setter- oder Getterfunktion, so hält das Programm an, wenn ein Zugriff erfolgt. Über den Callstack kann dann der Aufruf nachverfolgt werden.
		`int iVal1 = 0;  //without encapsulation`
		
		`static int iVal2 = 0; // with encapsulation`
		
		`int GetVal2(void)`
		`{`
		`  return iVal2;`
		`}`
		
		`void SetVal2(int iNewVal2)`
		`{`
		`  if (iNewVal2 < 4711)`
		`  {`
		`  iVal2 = iNewVal2;`
		`  }`
		`}`

A12	Für komplexe Datentypen, Variablen und Funktionen sind aussagekräftige Namen zu vergeben.
	Code wird lesbarer und ersetzt ggf. einen Kommentar.
		`int GetGreatestValue(int iValA, int iValB);`
		
		`BOOL IsValueNegativ(int iValue);`

A13	Es ist der höchste Warninglevel einzustellen. *Warnungen weisen oft auf mögliche Programmierfehler hin.* *„Der Compiler ist dein Freund.“*	**Microsoft Visual Studio:** *Projekt mit rechter Maustaste anwählen → Properties → C/C++ → General → Warning Level* auf *Level4 (/W4)* setzen. **Keil uVision 5: ARM Compiler 6** Project → *Options for Target* → C/C++(AC6) → *Warnings:* auf *AC5-like Warnings* setzen. All Warnings führt zu Fehlern im BSP **STM32CubeIDE:** *Projekt mit rechter Maustaste anwählen -> Properties -> C/C++ Build -> Settings -> MCU GCC Compiler -> Warnings -> Enable all warnings about questionable constructions (-Wall)*
A14	Warnungen müssen wie Fehler gehandhabt werden. *Somit ist man gezwungen, dass alle Warnungen behandelt werden. Dies führt allerdings dazu, dass man den Code entsprechend anpassen muss.* *„Versuche nie Menschen zu ändern, ändere die Prozesse die auf die Menschen zugreifen“.*	**Microsoft Visual Studio:** *Projekt mit rechter Maustaste anwählen → Properties → C/C++ → General → Treat Warnings As Errors auf Yes (/WX) setzen* **Keil uVision 5: ARM Compiler 6** *Options for Target* → C/C++(AC6) → Turn Warnings into Errors anwählen. **STM32CubeIDE:** *Projekt mit rechter Maustaste anwählen -> Properties -> C/C++ Build -> Settings -> MCU GCC Compiler -> Warnings -> Treat warnings as errors (-Werror)*

Code-Layout		

CL1	Die öffnende { und schließende } geschweifte Klammer beginnt stets in einer neuen Zeile.	`for (ui = 0; ui < 10; ui++)`
		`{`
		`  // Code`
		`}`

CL2	Innerhalb der geschweiften Klammern ist der Codeblock um **drei** Leerzeichen einzurücken. Dies gilt auch nach case-Labels in einer switch/case-Kontrollstruktur, bei der Deklaration von komplexen Datentypen und ihren Elementen, sowie bei Kommentaren. (siehe Regel A1: Umwandlung von Tabs in Leerzeichen)	`struct SyncMechanism`
		`{`
		`  long int liTimeStamp;`
		`  int iNumberOfIntervals;`
		`};`

CL3	Nach den Schlüsselworten if, while, for, switch folgt ein Leerzeichen.	`for ()`
		`if ()`
		`while ()`

CL4	Nach einem Funktionsnamen folgt unmittelbar die öffnende runde Klammer ((Deklaration, Definition, Aufruf).
	Daran erkennt man, dass dies zusammengehört.
		`Good:`
		`Call250usTask();`
		`Bad:`
		`Call250usTask  ();`

CL5	Vor oder nach den primären Operatoren ((), ->, . und [ ]) oder zwischen unären Operatoren und ihrem Operanden sowie und dem logischen ! sind **keine** Leerzeichen zu verwenden.
	Vor und nach den relationalen (<, <= ,> ,>= ,== , !=), arithmetischen (*, /, +, -, %) und logischen Operatoren (&&, ||) sowie bei bitorientierten Operationen (&, |, ^, ~, <<, >>) ist jeweils **ein Leerzeichen** einzufügen. Bei einer Zuweisung ist ebenso vor und nach dem = ein Leerzeichen einzufügen (gilt ebenso für Kurzformen). Zur Erhöhung der Lesbarkeit kann aber bei komplexeren Ausdrücken von dieser Regel abgewichen werden (Hinzfügen von Leerzeichen).
		`psAddress->iZipCode = asAddressOld[3].iZipCode; //primary`
		`iNumbers++; //unary (++)`
		`uiVal1 = uiVal2 >> 2U;`
		`iVal3 = (4 * iVal4) + 4;`
		
		`if ((iA < 10) && (iA != iC))`
		
		`if (LPC_TIM0->IR & (1<<2))`

CL6	Die geschweiften Klammern { } stehen bei switch, while, for, do/while, if, else, else/if direkt unter dem ersten Buchstaben des Schlüsselwortes.	`switch (iSelection)`
		`{`
		`  case 10:`
		`  while (i < 10)`
		`  {`
		`  i = GetStatusOfPort();`
		`  }`
		`  break;`
		`  case 20: // fall through is ok here`
		`  case 30:`
		`  // commands if iSelection is 20 or 30`
		`  break;`
		`  default:`
		`  break;`
		`}`

Datentypen, Variablen, Konstanten und Ausdrücke		

DV1	Variablen und Konstanten haben selbstsprechende Namen in Englisch und beginnen mit einem Großbuchstaben. Der Datentyp soll dabei als **Präfix** (Kurzform) dem Namen hinzugefügt werden.	`int iTrials = 1; //prefix is i`
		`const float cfPI = 3.1415f; // prefix is cf`

DV2
ES
	Es werden nicht mehr die nativen Datentypen, sondern die typedefs aus stdint.h (C99) verwendet. Falls stdint.h nicht vorhanden ist, ist dies in einer eigenen Headerdatei abzubilden.
	uint8_t, uint16_t, uint32_t, uint64_t für unsigned Variablen
	int8_t, int16_t, int32_t, int64_t für signed Variablen
	
	Für float, double und long double sind eigene typedefs zu verwenden:
	f32_t, f64_t, f96_t, f124_t
	long double ist plattformabhängig. Fließkommazahlen sollten aus Performanzgründen auf einem Mikrocontroller ohne FPU vermieden werden.
		`#include <stdint.h>`
		
		`//if stdint.h not available`
		`typedef unsigned int uint32_t;`
		
		`typedef float f32_t;`
		`typedef double f64_t;`
		`typedef long double f96_t;`

DV3
II
	Bei Variablennamen ist die **Ungarische Notation** als Präfix zu verwenden.
	short int — si — signed char — c
	unsigned short int — usi — unsigned char — uc
	int — i — float — f
	unsigned int — ui — double — d
	long int — li — long double — ld
	unsigned long int — uli — _Bool (C99 native) — b
	long long int — lli
	unsigned long long int — ulli
	In <stdbool.h> finden sich die Makros für bool, true und false, um die Kompatibilität mit C++ zu gewährleisten. Der C-Standard gibt bei char nicht vor, ob es sich um ein signed char oder ein unsigned char handelt. Daher sollte man immer signed oder unsigned als Modifizierer bei char verwenden. Bei short int, int, long int und long long int handelt es sich immer um signed Variablen.
		`short int siCounter; // counter 1`
		`unsigned short int usiCounter; // counter 2`
		`int iCounter; // counter 3`
		`unsiged int uiCounter; // counter 4`
		`long int liCounter; // counter 5`
		`unsigned long int uliCounter; // counter 6`
		`long long int lliCount; // counter 7`
		`unsigned long long int ulliCounter; // counter 8`
		`signed char cByte; // a signed byte`
		`unsigned char ucByte; // an unsigned byte`
		`float fAverage; // Average 1`
		`double dAverage; // Average 2`
		`long double ldAverage; // Average 3`
		`_Bool bErrorflag1; // Errorfl.1 (C99 native)`
		`bool bErrorflag2; // Errorfl.2 (stdbool.h)`

DV3
ES
	Bei Variablennamen ist die **Kurzkennung** als Präfix zu verwenden.
	int16_t — s16 — int8_t — s8
	uint16_t — u16 — uint8_t — u8
	int32_t — s32 — f32_t — f32
	uint32_t — u32 — f64_t — f64
	int64_t — s64 — f96_t — f96
	uint64_t — u64 — _Bool (C99 native) — b
	In <stdbool.h> finden sich die Makros für bool, true und false, um die Kompatibilität mit C++ zu gewährleisten.
		`int16_t s16Counter;`
		`uint16_t u16Counter;`
		`int32_t s32Counter;`
		`uint32_t u32Counter;`
		`int64_t s64Counter;`
		`uint64_t u64Counter;`
		
		`int8_t s8Byte;`
		`uint8_t u8Byte;`
		`f32_t f32Average;`
		`f64_t f64Average;`
		`f96_t f96Average;`
		`// C99 native boolean`
		`_Bool bErrorflag1;`
		`// stdbool.h`
		`bool bErrorflag2;`

DV4	Pro Zeile ist immer nur eine Variable/Konstante zu deklarieren/definieren.	`int* pi1, pi2;  // not okay`

DV5	Bei der Definition einer Zeiger-Variablen/-Konstanten ist zwischen dem Datentyp und dem * kein Leerzeichen einzufügen.	`int * piVal1;  // not okay`
		`int* piVal2;  // okay`

DV6	Bei Konstanten ist noch ein c voranzustellen. Bei Pointern ist p und bei Arrays ist ein a dem Präfix voranzustellen. Einem void-Pointer ist ein pv als gesamter Präfix voranzustellen (Sonderrolle, keine void-Variable möglich)	`const float cfPI = 3.1415F;`
		`int aiSemester[7];`
		`int* piSum;`
		`char* pacSymbols[];`

DV7	Für Laufvariablen können ausnahmsweise i, j, k, l, m und n verwendet werden.	`int i;`
DV8	long-Konstanten sind mit einem L, unsigned Konstanten mit einem U und unsigned long-Konstanten sind mit einem UL abzuschließen. Keine Kleinbuchstaben u und l!	`lNumberOfBits = 221214233L;` `ulprojectcode = 994943848UL;`

DV9	**Fließkommakonstanten** haben immer mindestens eine Ziffer vor und nach dem Dezimalpunkt. Ein F signalisiert den Datentyp float, ohne F ist die Konstante vom Typ double. Der Kleinbuchstabe f ist **nicht** zu verwenden.	`const float cfPi = 3.1415F;`
		`const float cfGoldenCut = 0.618F;`
		`const double cdEuler = 2.7182818284;`

DV10	**Hexadezimalkonstanten** beginnen immer mit 0x (kein großes O!!!). Für A-F sind Großbuchstaben zu verwenden.	`const unsigned short int cuiValue = 0x1AF3;`
DV11	**Oktalkonstanten** (führende 0) sollten nicht verwendet werden.	`unsigned int uiVal = 072; //072 is octal constant`

DV12	Wenn Modifizierer verwendet werden (long, short, signed, unsigned), müssen immer die Basistypen (int, float, double, char) angegeben werden.	`long int liNumberOfBytes;  //okay`
		`long liRange;  // not okay`

DV13	Implizite **Casts** (casts durch Compiler) bei Zuweisungen sind zu vermeiden! Nur explizite casts verwenden, um zu zeigen, dass dies so gewollt ist.	`iVal = fVal;  // implicit cast – not okay`
		`iVal = (int)fVal;  // explicit cast - okay`

DV14	**Zeiger** sind vor ihrer ersten Verwendung mit NULL zu initialisieren, sofern dem Zeiger nicht in einer ersten Definition eine gültige Adresse zugewiesen wird.
	Wenn dies durchgängig erfolgt (mit Regel DV16) kann immer festgestellt werden, ob ein Zeiger gültig ist. NULL ist als ((void)0) definiert: #define NULL ((void*)0)*
		`int* piMemory = NULL;`
		
		`// check if pointer is valid`
		`if (piMemory != NULL)`

DV15	Bei **malloc** immer den sizeof-Operator (keine absoluten Werte) verwenden. *malloc liefert einen Zeiger vom Type **void*** zurück. Expliziter cast in C++ notwendig*	`piMemory = (int*)malloc(100 * sizeof(int));`
DV16	Nach einem free ist der Zeiger ebenso auf NULL zu setzen, um diesen dann wieder als ungültig zu kennzeichnen.	`free(piMemory);` `piMemory = NULL;`

DV17	**Eingebettete Zuweisungen** sind zu vermeiden. Eingebettete Zuweisungen sind entsprechend aufzuspalten.
	Der Programmcode wird dadurch lesbarer und ist auch weniger fehleranfällig.
		`//not okay`
		`if ((iTotal = GetTotal()) == 10)`
		`{`
		`  printf("do it\n");`
		`}`
		
		`// okay`
		`iTotal = GetTotal();`
		`if (iTotal == 10)`
		`{`
		`  printf("do it\n");`
		`}`

DV18	Signed und unsigned Variablen/Konstanten dürfen nicht in einer Anweisung/Bedingung verwendet werden.	`if (uiVal1 > iVal2) //possible wrong behaviour`

DV19	Die bitorientierten Operatoren (>> und <<) dürfen nicht auf signed Variablen angewendet werden.	`iVal2  = iVal2>>1U;  //not okay if iVal2 is negativ`
		`uiVal3 = iVal3>>1U;  //okay`

Komplexe Datentypen		

KD1	Variablen von komplexen Datentypen enum, union, struct, bitfield wird als **Präfix** vorangestellt:
	enum e
	union u
	struct s
	bitfields bf
		`enum States eProgrammStates;`
		`union MeasureTime uLastTime;`
		`struct Address sCustomerAddress;`
		`struct Flags bfNewFlags;`

KD2	Auch bei den komplexen Datentypen gilt:
	Bei Konstanten ist noch ein c voranzustellen. Bei Arrays ist ein a und bei Pointern ein p dem Präfix voranzustellen.
		`const enum States ceProgrammStates;`
		`struct Address asCustomerAddress[100];`
		`struct Address* psCustomerAddress;`
		`const struct Name csDefaultName = {"Hans","Mustermann"};`

KD3	Um Schreibarbeit zu sparen, sind von jedem obigen komplexen Datentyp (KD1) ein Typedef auch für einen Zeiger darauf zu deklarieren.
	Bei komplexen Datentypen erspart man sich dann bei den Definitionen und den Deklarationen die Verwendung der Schlüsselworte struct, union und enum.
		Kurzform
		`typedef struct Address`
		`{`
		`//elements of struct Address`
		`}Address_t;`
		
		`typedef Address_t* pAddress_t;`
		
		Langform
		`struct Address`
		`{`
		`//elements of struct Address`
		`};`
		`typedef struct Adress Address_t;`
		`typedef Address_t* pAddress_t;`
		
		`Address_t sMyAddress = {"Badstrasse 24",77652,"Offenburg"};`
		`pAddress_t psMyAddress = &sMyAddress;`

KD4	Eigenen Typdefinitionen (typedef) ist als Postfix ein _t nachzustellen.	`typedef float f32_t;`
KD5	Auf die Elemente eines Bitfelds sollte nie über eine Maske zugegriffen werden. *Es ist abhängig vom Compiler, wo die Elemente des Bitfelds innerhalb des Speicherbereichs angelegt werden.*	`if (bfstate&0x80000000) // not okay`

KD6	Die Elemente eines Bitfeldes müssen vom Typ unsigned int (II) oder uint32_t (ES) sein.	`struct BitfieldAPSR`
		`{`
		`  unsigned int uiVal1 : 1;`
		`  unsigned int uiVal2 : 3;`
		`};`

KD7	Bei der Deklaration einer Enumeration ist beim ersten Element der Startwert anzugeben, auch wenn dieser 0 ist (Default).	`enum States {Start = 0, Run, End};`
KD8	Bei Funktionszeigern sollte immer mit einem typdef gearbeitet werden.	`typedef (void)(*pf)(int, char, int*);`
KD9 ES	Rückgabewert und Übergabewert sind dabei nach DV3 im neuen Namen zu hinterlegen. Ebenso ist als Postfix ein _t nachzustellen.	`typedef (uint32_t)(*pf_u32_s32-s8_t)(int32_t, int8_t);`

Funktionen		

F1	Nach der öffnenden geschweiften Klammer werden alle lokalen Variablen pro Zeile einzeln definiert und immer initialisiert. Die zum Funktionsrumpf gehörenden geschweiften Klammern stehen dabei immer in der ersten Spalte und in einer neuen Zeile.	`char* CopyStringWithoutBlanks(char* pcDest, char* pcSource)`
		`{`
		`  // points to the result`
		`  char* pcDestination;`
		
		`  // number of chars`
		`  int iNumberOfChars = 0;`
		
		`  // here the implementation starts`
		
		`  return pcDestination;`
		`}`

F2	Funktionen sind vor dem Aufruf zu deklarieren. *Ansonsten greift die implizite Int-Regel (Returnwert wird als int angenommen).*	

F3	Bei der Funktionsdeklaration sind die optionalen Variablennamen anzugeben, falls Parameter übergeben werden. Werden keine Parameter übergeben, so ist bei der Deklaration ein void hinzuzufügen.
	Dies dient dem besseren Verständnis beim Lesen der Funktionsdeklaration.
		`char* CopyStringWithoutBlanks(char* pcDest, char* pcSource);`
		
		`int GetNumberOfTrials(void);`

F4	Es ist nur ein return pro Funktion erlaubt und zwar als letzte Anweisung.	
F5	Eine Funktion, die keinen Wert zurückgibt (void), hat keine return-Anweisung.	
F6	Die maximale Anzahl an Übergabeparameter sollte drei nicht überschreiten.	`int CalcMedian(int iVal1, int iVal2, int iVal3)`
F7	Eine Funktion darf nicht mehr als neun Bedingungen (in if, while, do..while, for) enthalten. Die maximale zyklomatische Komplexität CC ist dann 10. *In Testwell CMT++ ist die zyklomatische Komplexität als v(G) ausgewiesen.*	`CC = Anzahl Bedingungen + 1`

F8	Formale Parameter, die innerhalb einer Funktion nur lesend benutzt werden, sind mit dem Schlüsselwort const zu übergeben
	
	Bei Zeigern auf Strukturen kann verfeinert werden:
	const beim Datentyp: Die Daten sind konstant (nicht veränderbar)
	const beim Zeiger: Der Zeiger ist konstant.
		`unsigned int CalcSpeed(const unsigned int cuiVal)`
		`{`
		`  return (10 * cuiVal);`
		`}`
		`unsigned int calcTorque(const struct Vals* const cpcs)`
		`{`
		`  return (ps->uiKm * ps->uiI0);`
		`}`
		`//Alternative version of calcTorque`
		`//First const can be set after datatyp and before *`
		`unsigned int calcTorque(struct Vals const * const cpcs)`

F9
ES
	Strukturen dürfen nie per value übergeben werden.
	Diese benötigen sonst zu viel Speicher auf dem Stack.
		`int32_t CalcMedian(struct VeryBig smybig);  //not okay`
		`int32_t CalcMedian(struct VeryBig* const pcsmybig); //okay`

F10	Funktionen, die nur innerhalb eines Moduls (c-Datei) verwendet werden, sind als *static* (*Kapselungsprinzip*) in der c-Datei zu deklarieren. Ein Zugriff von außerhalb des Moduls (c-Datei) ist dann nicht mehr möglich.	`static int GetSum(int ival1, int ival2);`

F11 II	Es sind selbstsprechende Funktionsnamen zu verwenden. Der Funktionsname setzt sich aus einem Verb und weiteren spezifizierenden Wörtern in englischer Sprache zusammen. Verb und spezifizierende Wörter beginnen jeweils mit einem Großbuchstaben (Kein Camelcase). Typische Verben sind hierbei „Get“, „Set“, „Init“, „Delete“, „Getnext“, „Getlast“, „Print“, „Is“, … .	`char* CopyStringWithoutBlanks(char* pcDest, char* pcSource)`

F11
ES
	Es sind selbstsprechende Funktionsnamen zu verwenden. Der Funktionsname setzt sich wie folgt zusammen:
	<Kurzkennung Rückgabewert><Modulname><_><Verb><weitere spezifizierende Wörter>. Der Modulname ist der Name der C-Datei, in welcher die Funktion implementiert ist. Darauf folgen ein Unterscore, ein Verb und weitere spezifizierende Wörter in englischer Sprache. Verb und spezifizierende Wörter beginnen jeweils mit einem Großbuchstaben. Typische Verben sind hierbei „Get“, „Set“, „Init“, „Delete“, „Copy“, „GetNext“, „GetLast“, „Print“, „Is“, … .
	
	Für die Kurzkennung der Rückgabewerte sind zu verwenden:
	int16_t s16 int8_t s8
	uint16_t u16 uint8_t u8
	int32_t s32 f32_t f32
	uint32_t u32 f64_t f64
	int64_t s64 f96_t f96
	uint64_t u64 _Bool (C99 native) b
	void v bool (stdbool.h) b
	
	Falls Pointer zurückgegeben werden, ist der Kurzkennung noch ein p voranzustellen.
	Werden eigene signed Datentypen (typedefs) zurückgegeben, so ist ein **x** als Präfix zu verwenden. Falls es sich um einen eigenen unsigned Datentyp handelt, so ist ein **ux** als Präfix zu verwenden.
	Entsprechend sind Funktionsnamen in FreeRTOS aufgebaut, welches in Embedded Systems 2 zur Anwendung kommt.
	Ausnahme: Namen der ISR (Interrupt Service Routinen). Deren Namen sind in der startup.s in der Interruptvektortabelle schon vorgegeben.
		`// File PWM.c`
		
		`uint32_t u32PWM_GetSignalGPIO (uint8_t u8SignalSource);`
		
		`psPWMConfig pxPWM_GetConfig (uint8_t u8SignalSource);`
		
		`void SysTick_Handler(void)`

F12	Die main-Funktion ist immer die erste Funktion in der C-Datei Main.c.	

F13	Werden an eine Funktion Zeiger übergeben, so sind diese am Anfang der Funktion auf != NULL zu überprüfen (**Sanity Check**) und es ist ein korrektes Fehlerhandling zu implementieren.	`void DoCallByReference(int* piA)`
		`{`
		`  if (piA != NULL)`

Kontrollstrukturen			
K1	Sprünge mit **goto** sind nicht erlaubt. Entsprechend sind Labels außer bei switch/case-Kontrollstrukturen nicht notwendig.	`goto Label1; // not okay`	
K2	**break** ist nur in switch/case-Kontrollstrukturen erlaubt.		
K3	Eine switch/case-Kontrollstrukturen hat immer ein default als letzter Case (Fehlerausgabe etc.). Dieser letzte Case enthält auch ein break.	`default:` `  break;`	
K4	Ein **fallthrough** (fehlendes break) in einer switch/case-Kontrollstruktur ist ausführlich zu dokumentieren.	`case 1: // fallthrough to case 2, because same behaviour` `case 2:`	

K5	Das Auslassen von Teilen des Schleifenkörpers mittels **continue** ist nicht erlaubt.
	Dies kann durch einen angepassten Kontrollfluss erreicht werden.
		`//not okay`
		`while (iStatus < 10)`
		`{`
		`  iStatus = GetStatus();`
		`  if (iStatus == 4)`
		`  {`
		`  continue; // not okay`
		`  }`
		`  // other statements`
		`}`
			`//okay`
			`while (iStatus < 10)`
			`{`
			`  iStatus = GetStatus();`
			`  if (iStatus != 4)  //okay`
			`  {`
			`  // other statements`
			`  }`
			`}`

K6	Das Abbrechen des Programmes mit **exit()** ist nur in begründeten Ausnahmefällen (Sicherheit oder Notfall) möglich. Ansonsten ist exit generell zu vermeiden.	`if (iEmergency == 1)`
		`{`
		`  exit(0);`
		`}`	

K7	Die **bedingte Zuweisung ?:** (ternärer Operator) ist zu vermeiden, wenn dies verständlicher ausdrückt werden kann (als if-else-Abfrage). Ansonsten ist mittels Kommentar zu beschreiben, was mit der bedingten Zuweisung erreicht werden soll. *Die bedingte Zuweisung findet sich häufig in Funktionsmakros.*	`iZ = (iA > iB) ? iA : iB; // z = max(a, b)`	

K8	**elseif** ist aus Gründen der besseren Lesbarkeit **nicht** erlaubt!
	Dies kann einfach durch Klammerung {} und einem if ersetzt werden.
		`// not okay`
		`if (iVal == 1)`
		`{`
		`  iVal = 42;`
		`}`
		`elseif (iVal == 2)`
		`{`
		`  iVal = 79;`
		`}`
			`// okay`
			`if (iVal == 1)`
			`{`
			`  iVal = 42;`
			`}`
			`else`
			`{`
			`  if (iVal == 2)`
			`  {`
			`  iVal = 79;`
			`  }`
			`}`

K9	Ein Codeblock, der nur aus einer Zeile besteht, ist auch zu klammern.
	Der äußerste Codeblock eines case in einer switch/case-Kontrollstruktur ist nicht
		`if (iVal == 1)`
		`{`
		`  iVal = 42;`	

	zu klammern. *Ansonsten werden die sowieso schon umfangreichen (LOCpro) switch/case-Kontrollstrukturen noch länger.*	`}`
K10	Die Regeln der „**Strukturierten Programmierung“** sind einzuhalten. §1 Eine Funktion hat einen Eingang und einen Ausgang. §2 Nicht in Abfragen springen. §3 Nicht aus Abfragen herausspringen. §4 Nicht in Schleifen springen. §5 Nicht aus Schleifen herausspringen. *Mit Struktogrammen (Nassi-Shneiderman Diagrammen) können nur strukturierte Funktionen realisiert werden.*	
K11	Kurzformen (C-Standard: != 0 für wahr, 0 für falsch) sind zu vermeiden. Dieses implizite Wissen *(!= 0 für wahr)* sollte nicht vorausgesetzt werden. *Im C89 Standard gibt es noch kein true und false.*	`if (iVal)  // better: if (iVal != 0)`
K12	Ist die Anzahl der Schleifendurchläufe vor der Schleife bekannt, so ist die for- statt die while-Schleife zu verwenden. *While-Schleifen sind in der Regel fehleranfälliger und etwas schwerer lesbar.*	

K13	Schleifenvariablen (Laufvariablen) dürfen bei einer for-Schleife im Schleifenrumpf nicht verändert werden.	`for (i = 0; i < 100; i++)`
		`{`
		`  //more code`
		`  i = iUserInput; // not okay`
		`  //more code`
		`}`

K14	Fließkommavariablen (double und float) dürfen nicht als Schleifenvariablen (Laufvariablen) verwendet werden.
	Bei Fließkommazahlen gibt es Rundungsungenauigkeiten (IEEE754).
		`for (fVal = 0.0F; fVal < 10.0F; fVal += 0.01F) // not okay`
		
		`for (iVal = 0; iVal < 1000; iVal++) // okay`

K15	Fließkommavariablen (double und float) dürfen nie auf Gleichheit überprüft werden.
	Bei Fließkommazahlen gibt es Rundungsungenauigkeiten (IEEE754).
		`if (dVal == 3.5517) // not okay`
		
		`const double cdTolerance = 0.0001;`
		`double dDifference;`
		
		`dDifference = fabs(dVal - 3.5517);`
		
		`if (cdTolerance > dDifference) // okay`

Präprozessor		

P1	Jede Headerdatei (z.B. PWM.h) ist mittels include-Guard gegen ein mehrfaches Inkludieren zu schützen. Name der Headerdatei ist dabei wie rechts aufgezeigt zu verwenden.
	*Damit wird jede h-Datei bei der Übersetzung **einer** C-Datei nur einmal inkludiert. Oft werden h-Dateien über andere h-Dateien sonst mehrfach inkludiert.*
		`#ifndef _PWM_H`
		`#define _PWM_H`
		
		`// all Code`
		
		`#endif //_PWM_H`

P2	Eine c-Datei darf nie inkludiert werden.	
P3	**#defines** sind groß zu schreiben. *Damit lassen sich diese von Variablen besser unterscheiden.*	`#define NUMBEROFFILES 5`
P4	Werden defines zur bedingten Compilierung eingesetzt, so ist im Namen SW_ als Präfix vorzustellen.	`#define SW_LINUX` `#define SW_GLCD`

P5	Um Seiteneffekte zu minimieren sind Makros wie folgt zu implementieren
	• Das gesamte Makro ist zu klammern
	• Jeder Makroparameter ist zu klammern
	• Makroparameter und das gesamte Makro sind auf den korrekten Typ zu casten
		`//Swap the Bytes of a 16-Bit Value`
		
		`#define SWAPBYTES16(x) ((uint16_t)(((uint16_t)(x) << 8) |  \`
		`  ((uint16_t)(x) >> 8)))`

P6	Umfangreichere Makros sind lesbarer auf mehrere Zeilen aufzuteilen. Ein Backslash trennt die Makrozeilen voneinander.	`siehe P5`

P7	Um weitere Seiteneffekte auszuschließen, sollten inline-Funktionen Makros vorgezogen werden. Diese bieten eine bessere Typsicherheit und vermeiden unerwünschte Seiteneffekte.
	Inline-Funktionen erscheinen zwar im Sourcecode als Funktion, jedoch wird der Funktionsrumpf meist vom Compiler direkt an die Stelle des Funktionsaufrufes kopiert. Der Umfang des Maschinencodes wird dadurch größer – die Performanz nimmt zu. Um die inline-Funktionen auch außerhalb des Moduls aufrufen zu können, wird noch eine „richtige“ Implementierung zur Verfügung gestellt. Inline ist allerdings nur einen Hinweis an den Compiler. Umfangreiche Funktionen werden trotz inline Schlüsselwort nicht „geinlined“.
		`// MACRO: Sideeffect with char cval = 'Z' AND`
		`// usage with ISUPPER(cval++)`
		`#define ISUPPER(c) (((c) >= 'A') && ((c) <= 'Z'))`
		
		`// Inline function: No Sideeffect with char cval = 'Z' AND`
		`// usage with isUpper(cval++)`
		`#include <stdbool.h>`
		`inline bool isUpper(char cval)`
		`{`
		`  bool bret = false;`
		
		`  if ((cval >= 'A') && (cval <= 'Z'))`
		`  {`
		`  bret = true;`
		`  }`
		`  return bret;`
		`}`

P8	Symbolische Konstanten sind – falls möglich – durch konstante Variablen zu ersetzen.	`#define PI 3.1415  //avoid` `const double cdPi = 3.1415;  //better`
P9	Präprozessoranweisungen beginnen immer in der ersten Spalte der Zeile	`#ifdef __USE_GLCD`

Doxygen		
D1 ES	Von Anfang an ist **doxygen** zur Dokumentation einzusetzen für Dateiköpfe, Funktionen, Strukturen, Enumerationen, Unions und Bitfelder.	siehe www.doxygen.org
D2 ES	Jede Datei enthält einen **Dateikopf**, der von doxygen interpretiert werden kann. @file Name der Datei @brief Kurzbeschreibung des Moduls @author Name des Modulanlegers @date Erstellungsdatum	/** * @file MyFirstCFile.c * @brief File contains the main program * @author Daniel Fischer * @date 01.08.2010 * * Additional Infos: This module is certified... */
D3 ES	Jede Funktion enthält einen **Funktionskopf**, der von doxygen interpretiert werden kann. @fn Funktionsdeklaration @brief Kurzbeschreibung der Funktion @param Name und Beschreibung des formalen Paramters und nur falls vorhanden @return Datentyp und Beschreibung des Rückgabewerts, und nur wenn ungleich void @author Name des Programmierers @date Erstellungsdatum	/** * @fn int CalcGreatest(int iA, int iB) * @brief Calculates the greatest value * @param iA first value * @param iB second value * @return int greatest value of iA and iB * @author Daniel Fischer * @date 01.08.2010 * * The greatest value is calculated based on the * given two parameters. If both are equal... */
D4 ES	Doxygen ist so zu konfigurieren, dass *call graphs* und *include dependency graphs* grafisch angezeigt werden. *Dazu ist das Tool GraphViz zu installieren. Siehe hierzu die entsprechende Anleitung.*	

Priorität und Assoziativität		
PA1	Die Prioritäten der Operatoren sowie deren Assoziativität sind zu berücksichtigen.	**Priorität:** (niedrigerer Wert entspricht höherer Priorität) **Assoziativität** 1 ++(Postfix) --(Postfix) () [] -> . L nach R 2 ++(Präfix) --(Präfix) ! ~ -(unär) +(unär) &(Adresse) *(Indirektion) sizof type(Typumwandung/cast) R nach L 3 *(Multiplikation) / % L nach R 4 +(Addition) -(Subtraktion) L nach R 5 << >> L nach R 6 < <= > >= L nach R 7 == != L nach R 8 &(bitweise Und) L nach R 9 ^ L nach R 10 | L nach R 11 && L nach R 12 || L nach R 13 ?: R nach L 14 = *= /= %= += -= <<= >>= &= ^= |= R nach L 15 , L nach R

PA2	Der Kommaoperator (Priorität 15) darf nicht verwendet werden.	`int i,j,k;  // not allowed`
		`for (i = 0; i < 10 , j = 0 ; i++) // not allowed`

PA3	Der ternäre Operator ?: (Priorität 13) sollte nicht verwendet werden. Ist durch if-else zu ersetzen.	Siehe auch Regel K7
PA4	Es darf kein implizites Wissen über die Operatorpriorität und der Assoziativität vorausgesetzt werden. Ausdrücke mit verschiedenen Operatoren sind zu klammern.	`iA = ((iA + iB) / iC) * iB;` `if ((iA < 10) && (iB != iC))`

C++ Allgemein		

CPPA1	Es gilt DV3 ES sowie F11 ES
	Die meisten Studierenden werden später C++ im Umfeld von Embedded Systems einsetzen.
		`uint32_t u32Val1 = 73U;`
		`int32_t s32Val2 = 42;`
		`f32_t f32Val3 = 44.1F; //typedef notwendig`

CPPA2	Referenzen wird ein r vorangestellt. *Dadurch wird by Reference ersichtlich.*	`uint32_t& ru32Val = u32X;`

CPPA3	Es sind durchgängig Namensräume (namespace) zu verwenden.
	So ist sichergestellt, dass es zu keinem Namenskonflikt kommt, wenn eine importierte Bibliothek die gleichen Namen verwendet.
	Auf Elemente eines Namensraum wird mit qualifiziertem Namen zugegriffen.
		`namespace Area51`
		`{`
		`  /* classes, functions, … */`
		`}`
		`std::cout << "hello" << std::endl;`

CPPA4	Für die C-Standbibliothek sind die korrespondierenden Header zu verwenden.	`#include <cstring>`

CPPA5	Ranged based loops und auto ranged based loops sind zu präferieren.	`for (u32C : au32Array)`
		`{`
		`  std::cout << u32C << std::endl;`
		`}`

CPPA6	Variablen mit blockorientierter Gültigkeit können verwendet werden, falls sichergestellt ist, dass der Programmcode nie mit C89 übersetzt wird.
	C++ Funktionen sind i.d.R. deutlich kürzer als C-Funktionen. Daher ist die Zuordnung noch gegeben.
		`//Most common is the usage inside a loop`
		`for (uint32_t u32C = 0U; u32C < 10U; u32C++)`
		`{`
		`  ; //do something`
		`}`
		`uint32_t au32Array[] = { 0U, 42U, 73U, 97U };`
		`for (auto aVar : au32Array)`
		`{`
		`  std::cout << aVar << std::endl;`
		`}`

CPPA7	In Structs, Unions und Enums ist s, u und e voranzustellen.
	Da dies in C++ neue Datentypen sind, kann am Namen der Typ erkannt werden.
		`struct sConfig`
		`{`
		`  uint32_t u32NumberOfIniFiles;`
		`  uint32_t u32MaxEntriesInFile = 200U;`
		`};`
		`sConfig sConfig1 = { 0 };`

CPPA8	Enumerationen sollten in C++ nur als Scoped Enums realisiert werden. *Dadurch werden Namenskonflikte bei mehreren Enums vermieden (gleiche Elemente)*	`enum class eColors {Red, Orange, Blue, White};`
CPPA9	Objekte sind nur mit Braces zu initialisieren! Brace yourself! *Damit wird z.B. „Most Vexing Parse“ verhindert.*	`int32_t s32Val { 7 };` `Car myCar { 130U };`

CPPA10	Autotypen sind dort einzusetzen, wo diese sinnvoll sind (sich ändernde Datentypen – generische Programmierung). Variablen von auto-Typen erhalten ein a als Präfix.
	Machen auch Refactoring einfacher. Insbesondere bei Iteratoren einfacher.
		`auto aVar6{ 10.0 };  // f64_t`
		`auto aVar7{ false };  // bool`

C++ Klassen		

CPPK1	Bei der Klassendeklaration ergibt sich von oben nach unten die folgende Reihenfolge: public, protected, private.
	So ist sofort die Schnittstelle nach außen sichtbar.
	
	Klassennamen (selbstsprechend) beginnen mit einem Großbuchstaben.
	
	Attribute sollten möglichst private sein. Zugriff erfolgt über Getter- und Setter-Methoden.Bei den Methoden sind zuerst die Konstruktoren und dann der Destruktor zu aufzuführen.
		`class Something`
		`{`
		`  **public:**`
		`  //methods and class variables`
		`  static uint32_t u32NumOfInstances_c;`
		`  **protected:**`
		`  //methods`
		`  **private:**`
		`  //methods and attributes`
		`  uint32_t u32Average_;`
		`};`

CPPK2	Structs als Datencontainer (Plain Old Dataobject) können noch verwendet werden. Das Keyword struct ist bei Klassen zu vermeiden, da es unterschiedliche Default-Sichtbarkeiten gibt.	
CPPK3	Attributen (Membervariablen) haben ein _ (Underscore) am Ende des Namens. Klassenvariablen ist ein _c am Ende des Namens einzufügen.	`u32Average_` `u32NumOfInstances_c`
CPPK4	Die Nutzung der default-Konstruktoren und des default-Destruktors ist mit =default anzuzeigen.	`Empty(const Empty&) = default;`
CPPK5	Die Nichtnutzung der default-Konstruktoren und des default-Destruktors ist mit =delete anzuzeigen.	`SimpleString(const SimpleString&) = delete;`
CPPK6	Die C++ casts sind auch bei Klassen/Objekten vorzuziehen.	`Empty* pEmptyThing1 = static_cast<Empty*>(pCar1);`
CPPK7	Alle Objektmethoden, die keine Attribute beschreiben, sein mit const zu deklarieren/definieren.	`uint32_t u32GetMaxSpeed() const` `{ }`
CPPK8	RAII ist immer anzuwenden! ***RAII Resource Acquisition Is Initialization***	
CPPK9	Der Destruktor sollte immer virtuell sein! *Dies wird sich später bei Vererbung mit dem dynamischen Polymorphismus erst auszahlen.*	`virtual ~Car();`
CPPK10	Funktionen, die keine Exception werfen, sollten mit noexcept deklariert, definiert werden. *Es ergeben sich dadurch Vorteile bei der Codegenerierung.*	`Car(Car&& rCar) noexcept;`

ChangeLog:

21.08.18	3.0.0	Komplette Überarbeitung (Unterscheidung II/ES), weitere Regeln, weitere Kapitel (Präprozessor), Komplexe Datentypen	DF
30.09.18	3.0.1	Bugfixes	DF
03.10.18	3.0.2	Bugfixes, Codebeispiele hinzugefügt	DF
04.10.18	3.0.3	Zwei Bugfixes	DF
10.08.19	3.0.4	Diverse Bugfixes, Ergänzungen und Vereinheitlichung	DF
15.02.20	3.0.5	Diverse Bugfixes (NULL) und Einführung von „_“ bei Funktionsnamen in Embedded Systems, EMI, CC ergänzt, Blanks	DF
15.05.20	3.0.6	Redaktionelle Änderungen, CL6 zu PA verschoben, Kommentardichte anpasst, (A3)	DF
18.07.20	3.0.7	Anpassungen bezüglich Skriptneuerstellung Ing.-Inf. / Programmieren 2, Sanity Checks	DF
27.09.20	3.0.8	Neue Regeln KD7, P8 und redaktionelle Anpassungen	DF
12.10.20	3.0.9	PA4 erweitert, KD8 neu	DF
26.02.21	3.0.10	c% auf 10-30%, DV9 angepasst, KD9 ergänzt (ES), Anpassungen auf ARM Compiler 6	DF
05.03.22	3.0.11	Präprozessor (P9) eingefügt, C++ Teil komplett neu erstellt	DF
16.10.23	3.0.12	STM32CubeIDE-Einstellungen ergänzt.	

ToDo/Under Consideration

- Formale Spezifikation erweitern für KD2 und DV6 insbesondere hinsichtlich const mit Zeigern, verschiedene c berücksichtigen